DARSHAN PRAJAPATI
MCA-2
ROLL NO: 24
SUBMISSION DATE: 01/04/2026

SHELL SCRIPT PRACTICAL ASSIGNMENT-2

---

1. Write a script that takes file name from user and display all line starting with a or b or c. (Use grep/sed)

```
# using grep

echo "Enter file name:"
read filename

if [ -f "$filename" ]; then
   echo "Lines starting with a, b, or c:"
   grep '^[abc]' "$filename"
else
   echo "File not found!"
fi

# using sed

echo "Enter file name:"
read filename

if [ -f "$filename" ]; then
   echo "Lines starting with a, b, or c:"
   sed -n '/^[abc]/p' "$filename"
else
   echo "File not found!"
fi
```

2. Write a script that takes file name from user and display all line starting not with a or b or c. (Use grep/sed)

```
# using grep

echo "Enter file name:"
read filename
```

```
if [ -f "$filename" ]; then
   echo "Lines NOT starting with a, b, or c:"
   grep '^[^abc]' "$filename"
else
   echo "File not found!"
fi

# using sed
echo

echo "Enter file name:"
read filename

if [ -f "$filename" ]; then
   echo "Lines NOT starting with a, b, or c:"
   sed -n '/^[^abc]/p' "$filename"
else
   echo "File not found!"
fi
```

3. Write a script that takes file name from user and display all line starting not with a or b or c with line numbers. (Use grep/sed)

```
# using grep

echo -n "Enter file name:"
read filename

if [ -f "$filename" ]; then
   echo "Lines NOT starting with a, b, or c (with line numbers):"
   grep -n '^[^abc]' "$filename"
else
   echo "File not found!"
fi

# using sed

echo "Enter file name:"
```

```
read filename

if [ -f "$filename" ]; then
   echo "Lines NOT starting with a, b, or c (with line numbers):"
   sed -n '/^[^abc]/=' "$filename" | paste - - | awk '{print $1 ": " $2}'
else
   echo "File not found!"
fi
```

4. Write a script that takes file name from user and substitute all spaces " " with # value. (Use grep/sed)

```
#using sed

echo "Enter file name:"
read filename

if [ -f "$filename" ]; then
   echo "Output after replacing spaces with #:"
   sed 's/ /#/g' "$filename"
else
   echo "File not found!"
fi
```

5. Write a script that takes file name from user and display all line start with t or T and second character must be either 'h' or 's'. (Use grep/sed)

```
# using grep

echo -n "Enter file name:"
read filename

if [ -f "$filename" ]; then
   echo "Lines starting with t/T and second char h or s:"
   grep '^[Tt][hs]' "$filename"
else
   echo "File not found!"
fi
```

```
# using sed

echo -n "Enter file name:"
read filename

if [ -f "$filename" ]; then
   echo "Lines starting with t/T and second char h or s:"
   sed -n '/^[Tt][hs]/p' "$filename"
else
   echo "File not found!"
fi
```

6. Write a script that takes file name from user and display all line start with space(‘ ‘) . (Use grep/sed)

```
# using grep

echo -n "Enter file name:"
read filename

if [ -f "$filename" ]; then
   echo "Lines starting with a space:"
   grep '^ ' "$filename"
else
   echo "File not found!"
fi

# using sed

echo -n "Enter file name:"
read filename

if [ -f "$filename" ]; then
   echo "Lines starting with a space:"
   sed -n '/^ /p' "$filename"
else
   echo "File not found!"
fi
```

7. Write a script which takes input from a file, with multiple records, as:
Firstname:middlename:lastname:address:city:pin:phone
and displays output as:
Record 1
Lastname middlename firstname
Address
City – pin
Phone
Record 2
Lastname middlename firstname
Address
City – pin
Phone
and so on, for all records.

```
echo "Enter file name:"
read filename

if [ -f "$filename" ]; then
   awk -F ':' '
   {
      print "Record " NR
      print $3 " " $2 " " $1
      print $4
      print $5 " - " $6
      print $7
      print ""
   }' "$filename"
else
   echo "File not found!"
fi
```

8. Write a script that shows usernames and no. of processes running for them.

```
echo "Username    No. of Processes"

ps -e -o user= | sort | uniq -c | awk '{print $2 "\t\t" $1}'
```

9. The book master file contains the fields book_no, book_name, author, dateofpurchase ,each field is separated by hypen. write a script for
(A) Add
(b) Modify
(C) Delete
from above file.

```
file="books.txt"

while true
do
   echo "----- MENU -----"
   echo "1. Add Record"
   echo "2. Modify Record"
   echo "3. Delete Record"
   echo "4. Exit"
   echo "Enter your choice:"
   read ch

   case $ch in

   1)
      echo "Enter Book No:"
      read bno
      echo "Enter Book Name:"
      read bname
      echo "Enter Author:"
      read author
      echo "Enter Date of Purchase:"
      read dop

      echo "$bno-$bname-$author-$dop" >> $file
      echo "Record Added!"
      ;;

   2)
      echo "Enter Book No to Modify:"
      read bno
```

```
        if grep -q "^$bno-" $file; then
            echo "Enter New Book Name:"
            read bname
            echo "Enter New Author:"
            read author
            echo "Enter New Date:"
            read dop

            sed -i "s/^$bno-.*/$bno-$bname-$author-$dop/" $file
            echo "Record Modified!"
        else
            echo "Record not found!"
        fi
        ;;

    3)
        echo "Enter Book No to Delete:"
        read bno

        if grep -q "^$bno-" $file; then
            sed -i "/^$bno-/d" $file
            echo "Record Deleted!"
        else
            echo "Record not found!"
        fi
        ;;

    4)
        exit
        ;;

    *)
        echo "Invalid Choice!"
        ;;

    esac
done
```

10. Write a shell script to display list of files which can be either

regular or directory along with number of links in ascending order of links.

```
echo "Files with number of links (ascending order):"

ls -l | grep -v '^total' | awk '{print $2, $9}' | sort -n
```

11. write a script to count number of vowels in file irrespective of case.

```
echo "Enter file name:"
read filename

if [ -f "$filename" ]; then
   echo "Number of vowels (case-insensitive):"
   grep -oi '[aeiou]' "$filename" | wc -l
else
   echo "File not found!"
fi
```

12. write a script that accepts a string followed by one or more file names from command line and display no of lines that consists of given string each file

```
# Check minimum arguments
if [ $# -lt 2 ]; then
   echo "Usage: $0 <string> <file1> [file2 ...]"
   exit 1
fi

pattern=$1
shift   # remove first argument (string), rest are files

for file in "$@"
do
   if [ -f "$file" ]; then
      count=$(grep -c "$pattern" "$file")
      echo "$file : $count"
   else
```

```
    echo "$file : File not found"
  fi
done
```

13. Create a text file with the headings and data as bill_no, cust_no, cust_name, address, city, pin, current_meter_reading, previous_meter_reading, month. Write a script that takes the input from this file and displays the bill for the query against cust_no/bill_no/cust_name. (input at least 15/20 records in this text file); otherwise message that no record exists

```
file="bill.txt"

echo "Search by:"
echo "1. Bill No"
echo "2. Customer No"
echo "3. Customer Name"
read choice

echo "Enter value:"
read value

case $choice in
1) result=$(grep "^$value:" $file) ;;
2) result=$(grep "^[^:]*:$value:" $file) ;;
3) result=$(grep -i ":$value:" $file) ;;
*) echo "Invalid choice"; exit ;;
esac

if [ -z "$result" ]; then
  echo "No record exists!"
else
  echo "$result" | while IFS=":" read bno cno cname addr city pin curr prev month
  do
    units=$((curr - prev))
    rate=5   # per unit
    bill=$((units * rate))

    echo "------ ELECTRICITY BILL ------"
    echo "Bill No: $bno"
```

```
        echo "Customer No: $cno"
        echo "Name: $cname"
        echo "Address: $addr"
        echo "City: $city - $pin"
        echo "Month: $month"
        echo "Units Consumed: $units"
        echo "Total Bill: Rs. $bill"
        echo "--------------------------------"
    done
fi
```

14. Write a shell script to generate summary from the sales.dat file.Sales made by 3 salesman by selling 3 products are entered in a file. Add atleast 10 records. The format is as shown below:
Salesman:Product1:Product2:Product3
Sample data:
Mr. Abhishek Sharma:21:29:12
Mr. Akash Srivastava:11:15:28
Mr. Abhilash Dwivedi:31:04:13
Calculate the followings :
1. Total sales of the company
2. Highest sold product
3. Best salesman (who sold the most)
4. Worst salesman (who sold the least)

```
file="sales.dat"

awk -F ':' '
BEGIN {
    total = 0
    p1 = p2 = p3 = 0
    max = -1
    min = 999999
}

{
    salesman = $1
    s1 = $2
    s2 = $3
```

```
    s3 = $4

    sum = s1 + s2 + s3

    total += sum

    p1 += s1
    p2 += s2
    p3 += s3

    if (sum > max) {
        max = sum
        best = salesman
    }

    if (sum < min) {
        min = sum
        worst = salesman
    }
}

END {
    print "Total Sales of Company:", total

    if (p1 > p2 && p1 > p3)
        print "Highest Sold Product: Product1"
    else if (p2 > p1 && p2 > p3)
        print "Highest Sold Product: Product2"
    else
        print "Highest Sold Product: Product3"

    print "Best Salesman:", best
    print "Worst Salesman:", worst
}
' "$file"
```

15. Write a shell script to generate summary from a file :
“student.dat” with following format :
Student_no : student_name : gender : marks1 : marks2 :
marks3
Each field must be separated by a delimeter ‘-‘

Process the following queries:

1. Calculate the total marks of each student

2. Calculate the percentage of marks for each student

3. Count the total number of male and female students

4. Count the total number of students who pass and those who fail.

```
file='student.dat'

awk -F '-' '
BEGIN {
   male = female = 0
   pass = fail = 0
   print "Student Summary "
   print "--------------------"
}
{
   total = $4 + $5 + $6
   percent = total/3

   print "Student:",$2
   print "Total marks:",total
   printf "Percentage:%.2f\n",percent

   if($3 == "M")
      male++
   else if ($3 == "F")
      female++

    if ($4 >= 40 && $5 >= 40 && $6 >= 40)
      pass++
   else
      fail++

   print "-----------------------------"
}
END {
     print "Total Male Students:", male
   print "Total Female Students:", female
   print "Total Pass Students:", pass
   print "Total Fail Students:", fail
```

```
}
' $file
```

16. Create a file employee.dat with fields:
emp_id:emp_name:department:basic_salary:hra:da:pf
(Insert at least 15 records)
Write a shell script to:
Calculate gross salary (basic + hra + da)
Calculate net salary (gross - pf)
Display details of employee based on emp_id / emp_name
Display highest paid employee

```
file="employee.dat"

echo "Search by:"
echo "1. Emp ID"
echo "2. Emp Name"
read choice

echo "Enter value:"
read value

awk -F ':' -v ch=$choice -v val="$value" '
BEGIN {
   max = -1
}

{
   emp_id = $1
   name = $2
   dept = $3
   basic = $4
   hra = $5
   da = $6
   pf = $7

   gross = basic + hra + da
   net = gross - pf

   # Find highest paid employee
```

```
    if (net > max) {
        max = net
        max_emp = name
    }

    # Search logic
    if ((ch == 1 && emp_id == val) || (ch == 2 && name == val)) {
        print "----- Employee Details -----"
        print "ID:", emp_id
        print "Name:", name
        print "Department:", dept
        print "Gross Salary:", gross
        print "Net Salary:", net
        print "----------------------------"
        found = 1
    }
}

END {
    if (found != 1)
        print "No record found!"

    print "Highest Paid Employee:", max_emp, "Salary:", max
}
' "$file"
```

17. Electricity Bill Summary
Create a file electricity.dat with fields:
consumer_no:name:units_consumed:month
Write a shell script to:
Calculate bill using slab:
0–100 → ₹2/unit
101–200 → ₹3/unit
Above 200 → ₹5/unit
Display total revenue collected
Find consumer with highest bill
Search record by consumer_no/name

```
file="electricity.dat"

echo "Search by:"
```

```
echo "1. Consumer No"
echo "2. Name"
read ch

echo "Enter value:"
read val

awk -F ':' -v choice=$ch -v value="$val" '
BEGIN {
   total_revenue = 0
   max_bill = -1
}

{
   cno = $1
   name = $2
   units = $3
   month = $4

   # Slab calculation
   if (units <= 100)
      bill = units * 2
   else if (units <= 200)
      bill = (100 * 2) + ((units - 100) * 3)
   else
      bill = (100 * 2) + (100 * 3) + ((units - 200) * 5)

   total_revenue += bill

   # Highest bill
   if (bill > max_bill) {
      max_bill = bill
      max_name = name
   }

   # Search
   if ((choice == 1 && cno == value) || (choice == 2 && name == value)) {
      print "----- Bill Details -----"
      print "Consumer No:", cno
      print "Name:", name
      print "Units:", units
```

```
        print "Month:", month
        print "Bill Amount: Rs.", bill
        print "------------------------"
        found = 1
    }
}

END {
    if (found != 1)
        print "No record found!"

    print "Total Revenue: Rs.", total_revenue
    print "Highest Bill Consumer:", max_name, "Amount:", max_bill
}
' "$file"
```

18. Library Management System
Create a file library.dat with fields:
book_id:title:author:student_name:issue_date:return_date
Write a shell script to:
Display all issued books
Find books issued to a particular student_name
Count total issued books
Display overdue books (assume current date input)

```
file="library.dat"

echo "Enter student name to search:"
read sname

echo "Enter current date (DD-MM-YYYY):"
read today

echo "----- All Issued Books -----"
cat "$file"

echo "----- Books issued to $sname -----"
grep -i ":$sname:" "$file"

echo "----- Total Issued Books -----"
```

```
wc -l < "$file"

echo "----- Overdue Books -----"

awk -F ':' -v today="$today" '
function to_days(d) {
   split(d, a, "-")
   return a[3]*365 + a[2]*30 + a[1]
}

BEGIN {
   today_days = to_days(today)
}

{
   return_date = $6
   r_days = to_days(return_date)

   if (r_days < today_days) {
      print "Book:", $2, "| Student:", $4, "| Return Date:", $6
   }
}
' "$file"
```

19. Product Inventory Analysis
Create a file inventory.dat with fields:
product_id:product_name:quantity:price
Write a shell script to:
Calculate total inventory value
Find product with highest quantity
Find product with lowest price
Search product by product_id/name

```
file="inventory.dat"

echo "Search by:"
echo "1. Product ID"
echo "2. Product Name"
read ch

echo "Enter value:"
```

```
read val

awk -F ':' -v choice=$ch -v value="$val" '
BEGIN {
   total_value = 0
   max_qty = -1
   min_price = 999999
}

{
   pid = $1
   pname = $2
   qty = $3
   price = $4

   value_prod = qty * price
   total_value += value_prod

   # Highest quantity
   if (qty > max_qty) {
      max_qty = qty
      max_product = pname
   }

   # Lowest price
   if (price < min_price) {
      min_price = price
      min_product = pname
   }

   # Search
   if ((choice == 1 && pid == value) || (choice == 2 && pname == value)) {
      print "----- Product Details -----"
      print "ID:", pid
      print "Name:", pname
      print "Quantity:", qty
      print "Price:", price
      print "Total Value:", value_prod
      print "---------------------------"
      found = 1
   }
```

```
}

END {
   if (found != 1)
      print "No record found!"

   print "Total Inventory Value:", total_value
   print "Highest Quantity Product:", max_product, "Qty:", max_qty
   print "Lowest Price Product:", min_product, "Price:", min_price
}
' "$file"
```

20. Cricket Score Analysis
Create a file cricket.dat with fields:
player_name:runs:balls:fours:sixes
Write a shell script to:
Calculate strike rate of each player
Find highest scorer
Count players scoring above 50
Display summary of all players

```
file="cricket.dat"

awk -F ':' '
BEGIN {
   max_runs = -1
   count50 = 0

   print "----- Player Summary -----"
}

{
   name = $1
   runs = $2
   balls = $3
   fours = $4
   sixes = $5

   # Strike Rate
   strike_rate = (runs / balls) * 100
```

```
    printf "Player: %s | Runs: %d | Balls: %d | SR: %.2f\n", name, runs, balls, strike_rate

    # Highest scorer
    if (runs > max_runs) {
        max_runs = runs
        top_player = name
    }

    # Count players above 50
    if (runs > 50)
        count50++
}

END {
    print "----------------------------"
    print "Highest Scorer:", top_player, "Runs:", max_runs
    print "Players scoring above 50:", count50
}
' "$file"
ss
```